

PEARLS AQI PREDICTOR

Comprehensive Project Report

10 Pearls Internship Capstone Project City: Sukkur, Sindh, Pakistan Date: June 2026

TABLE OF CONTENTS

1. Project Overview
2. Live Demo URLs
3. Technology Stack
4. System Architecture
5. Project Structure
6. Data Sources & Collection
7. Feature Engineering
8. Machine Learning Models & Performance
9. CI/CD Pipeline (GitHub Actions)
10. FastAPI Backend
11. React Frontend
12. MongoDB Feature Store
13. SHAP Explainability Analysis
14. Issues Faced & Solutions
15. Project Requirements Fulfillment

1. PROJECT OVERVIEW

The Pearls AQI Predictor is a complete end-to-end machine learning pipeline for predicting Air Quality Index (AQI) in Sukkur, Sindh, Pakistan for the next 3 days. It runs on a 100% serverless stack using GitHub Actions for fully automated data collection, feature engineering, model training, and real-time AQI predictions.

The system fetches real-time weather and pollution data from Open-Meteo and Open Weather APIs, engineers 23 features including temporal lag features, stores them in MongoDB Atlas as a feature store, and serves predictions through a FastAPI backend and a React web dashboard.

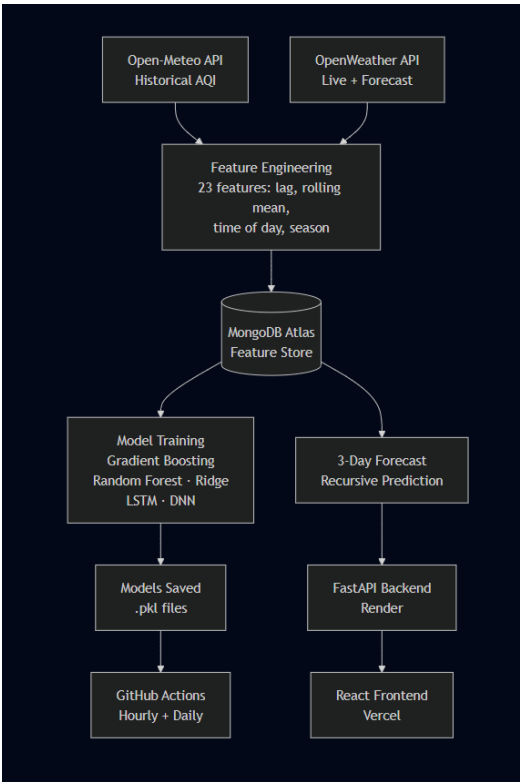
2. LIVE DEMO URLS

Service	URL
React Frontend	https://10pearls-aqi-predictor.vercel.app/
FastAPI Backend	https://pearls-aqi.onrender.com
API Documentation	https://pearls-aqi.onrender.com/docs

3. TECHNOLOGY STACK

Category	Technology
Language	Python 3.11+
Frontend	React + Vite + Recharts
Data APIs	Open-Meteo (historical AQI), OpenWeather (live + forecast)
Database	MongoDB Atlas (Feature Store)
ML Models	Gradient Boosting, Random Forest, Ridge Regression, LSTM, DNN
Explainability	SHAP
Backend API	FastAPI
Dashboard	Streamlit (reference)
CI/CD	GitHub Actions
Deployment	Vercel (frontend), Render (API)
Python HTTP	requests, certifi
Data Processing	pandas, numpy, scikit-learn, joblib

4. SYSTEM ARCHITECTURE



Data Flow Summary:

- Every hour: GitHub Actions fetches live AQI + weather → engineers lag features → pushes to MongoDB
- Every day: GitHub Actions retrains Ridge model with latest MongoDB data → saves updated .pkl + metrics
- On user request: React frontend calls FastAPI → FastAPI loads .pkl models → returns predictions

5. DATA SOURCES & COLLECTION

Data Sources

Source	Type	Coverage
Open-Meteo Air Quality API	Historical AQI, PM2.5, PM10, NO2, CO, O3	Last 3 months
Open-Meteo Archive API	Historical temperature, humidity, wind speed	Last 3 months
OpenWeather Current API	Live weather data	Real-time
OpenWeather Forecast API	5-day weather forecast	Next 3 days

Note: All AQI values use the European AQI scale (0–500+) from Open-Meteo, NOT the OpenWeather 1–5 scale.

Historical Data Fetch Process (data_fetch/fetch_historical.py)

The script fetches 90 days of hourly data by:

1. Calling Open-Meteo Air Quality API for PM2.5, PM10, NO2, CO, O3, and European AQI
2. Calling Open-Meteo Archive API for temperature, humidity, wind speed
3. Merging both datasets by timestamp
4. Saving merged records to MongoDB Atlas aqi_features collection

Coordinates used: **Latitude 27.7052°N, Longitude 68.8574°E (Sukkur, Pakistan)**

6. FEATURE ENGINEERING

23 Engineered Features

Feature	Description	Type
hour	Hour of day (0–23)	Time
day	Day of month	Time
month	Month of year	Time
temperature	Air temperature in °C	Weather
humidity	Relative humidity %	Weather
wind_speed	Wind speed in m/s	Weather
pm2_5	Fine particulate matter $\mu\text{g}/\text{m}^3$	Pollutant
pm10	Coarse particulate matter $\mu\text{g}/\text{m}^3$	Pollutant
no2	Nitrogen dioxide $\mu\text{g}/\text{m}^3$	Pollutant
co	Carbon monoxide $\mu\text{g}/\text{m}^3$	Pollutant
o3	Ozone $\mu\text{g}/\text{m}^3$	Pollutant
aqi_lag_1	AQI 1 hour ago	Lag
aqi_lag_3	AQI 3 hours ago	Lag
aqi_lag_24	AQI 24 hours ago	Lag
aqi_rolling_mean_24	24-hour rolling average AQI	Rolling
aqi_change_rate	Rate of AQI change (diff)	Derived
time_of_day	Morning/Afternoon/Evening/Night	Categorical
is_weekend	Weekend flag (0 or 1)	Binary
season	Winter/Spring/Summer/Autumn	Categorical

Total dataset: 23 features, 2160+ hourly records

Feature Engineering Pipeline (feature_pipeline/run_pipeline.py)

The hourly pipeline:

1. Fetches current weather from OpenWeather
2. Fetches current AQI from Open-Meteo European scale
3. Retrieves the last 24 records from MongoDB to compute lag features
4. Calculates: aqi_lag_1, aqi_lag_3, aqi_lag_24, aqi_rolling_mean_24, aqi_change_rate
5. Adds categorical features: time_of_day, is_weekend, season
6. Pushes new record to both aqi_features and aqi_engineered collections

8. MACHINE LEARNING MODELS & PERFORMANCE

Model Performance Results

Model	R ² Score	RMSE	MAE	Status	Verdict
Gradient Boosting	0.9917	1.2187	0.3714	Primary	No overfitting
Random Forest	0.9846	1.6580	0.5181	Secondary	No overfitting
Ridge Regression	0.8862	4.5046	2.5873	Secondary	No overfitting
Lasso	0.8833	4.5620	2.5094	Trained	No overfitting
ElasticNet	0.8789	4.6471	2.6446	Trained	No overfitting
LSTM	-24.49	67.358	61.343	Experimental	Overfitting

Primary model: Gradient Boosting (R² = 0.9917)

Model Files

File	Size	Description
gradient_boosting.pkl	323 KB	Primary prediction model
random_forest.pkl	1.97 MB	Ensemble secondary model
ridge.pkl	705 B	Linear secondary model
scaler.pkl	1.63 KB	StandardScaler for Ridge
label_encoder.pkl	575 B	Encodes time_of_day categories

Training Pipeline (training_pipeline/run_training.py)

The daily automated training:

1. Fetches all features from MongoDB aqi_features collection
2. Re-engineers lag features
3. Encodes categorical columns with LabelEncoder and StandardScaler
4. Retrains Ridge Regression model (alpha=1.0)
5. Evaluates R^2 , RMSE, MAE on training data
6. Saves updated ridge.pkl, scaler.pkl, label_encoder.pkl
7. Persists metrics to MongoDB model_metadata collection with timestamp

Training Notebooks

Notebook	Purpose
01_train_models.ipynb	Trains Gradient Boosting, Random Forest, Ridge, Lasso, ElasticNet, LSTM
02_shap_analysis.ipynb	Runs SHAP explainability analysis on Gradient Boosting
03_forecast.ipynb	Generates 3-day recursive forecast

8. CI/CD PIPELINE (GITHUB ACTIONS)

A) Feature Pipeline — Runs Every Hour

File: .github/workflows/feature_pipeline.yml **Schedule:** cron: '0 * * * *' (top of every hour)

Steps:

1. Checkout code from GitHub
2. Set up Python 3.10
3. Install: requests, python-dotenv, pandas, numpy, pymongo, certifi, scikit-learn, joblib
4. Create .env file from GitHub Secrets
5. Run python feature_pipeline/run_pipeline.py

B) Training Pipeline — Runs Every Day at Midnight

File: .github/workflows/training_pipeline.yml **Schedule:** cron: '0 0 * * *' (midnight UTC daily)

Steps:

1. Checkout code from GitHub
2. Set up Python 3.10
3. Install: same as above + shap
4. Create .env file from GitHub Secrets
5. Run python training_pipeline/run_training.py

GitHub Actions Secrets Required

OPENWEATHER_API_KEY — OpenWeather API key

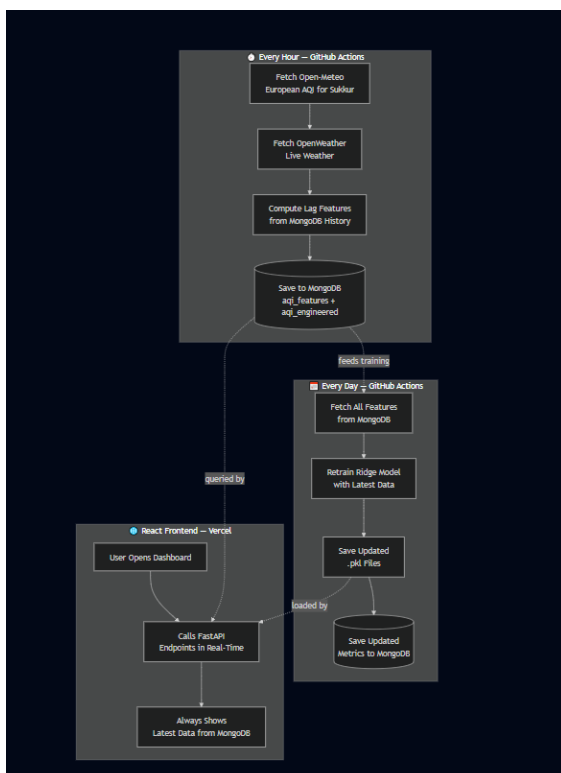
MONGODB_URI — MongoDB Atlas connection string

LAT — Latitude (27.7052)

LON — Longitude (68.8574)

CITY — City name (Sukkur)

Pipeline Flow



9. FASTAPI BACKEND

File: api/main.py **Deployed at:** <https://pearls-aqi.onrender.com> **Docs:** <https://pearls-aqi.onrender.com/docs>

API Endpoints

Endpoint	Method	Description
/	GET	Project info and available endpoints
/health	GET	Health check with timestamp
/current	GET	Current AQI and weather (European scale)
/forecast	GET	3-day forecast using Gradient Boosting
/forecast/{model}	GET	3-day forecast using selected model
/historical	GET	Historical data from MongoDB (default: last 100 records)
/models	GET	Available models and their metrics

Available Model Names

- gradient_boosting — Primary model ($R^2=0.9917$)
- random_forest — Secondary ($R^2=0.9846$) | ridge — Secondary ($R^2=0.8862$)

Forecast Logic

The /forecast/{model} endpoint:

1. Loads requested model from .pkl
2. Fetches MongoDB AQI history for lag computation
3. Fetches OpenWeather 5-day forecast (3-hourly intervals)
4. Fetches today's pollution data from Open-Meteo
5. Performs **recursive prediction** for next 72 hours — each prediction feeds into the next as aqi_lag_1
6. Aggregates hourly predictions into 3-day daily averages
7. Returns both daily_forecast (3 items) and hourly_forecast (up to 24 items)

10. REACT FRONTEND

Deployed at: <https://pearls-aqi-frontend.vercel.app> **Framework:** React 19 + Vite 8 + React Router DOM 7 + Recharts 3

Pages

Dashboard (/)

- Current AQI card showing temperature, humidity, wind speed
- Model selector (Gradient Boosting / Random Forest / Ridge)
- 3-day forecast cards with color-coded AQI categories
- 72-hour AQI trend line chart (Recharts LineChart)

History (/history)

- Stats row: average, max, min, total records
- Daily average AQI trend chart
- Temperature trend chart
- PM10 and PM2.5 trend chart
- Monthly average AQI bar chart

Analysis (/analysis)

- SHAP feature importance bar chart
- Current pollutant levels bar chart
- All model performance metrics table (R^2 , RMSE, MAE)

Alerts (/alerts)

- Dynamic health warning box based on 3-day forecast maximum AQI
- Contextual recommendations by AQI severity level
- Full European AQI scale reference table

12. MONGODB FEATURE STORE

Provider: MongoDB Atlas (Cloud) **Database:** pearls_aqi

Collections

Collection	Contents
aqi_features	Raw hourly records: AQI, pollutants, weather
aqi_engineered	Fully engineered records with all 23 features
model_metadata	Training run logs: R ² , RMSE, MAE, timestamp

13. SHAP EXPLAINABILITY ANALYSIS

SHAP (SHapley Additive exPlanations) was run on the Gradient Boosting primary model to explain feature contributions.

Top Feature Importance (SHAP Values)

Rank	Feature	SHAP Value
1	aqi_lag_1	9.03
2	aqi_change_rate	0.75
3	aqi_lag_3	0.50
4	aqi_rolling_mean_24	0.08
5	aqi_lag_24	0.02

Key Insight: The previous hour's AQI is by far the strongest predictor (SHAP value 9.03 — more than 12x the next feature). Sukkur's air quality follows strong temporal autocorrelation patterns driven primarily by dust and particulate matter. Weather features (temperature, humidity, wind) contribute very little, confirming that **dust, not weather is the primary pollution driver in Sukkur.**

14. ISSUES FACED & SOLUTIONS

Issue 1: scikit-learn Version Mismatch (Streamlit Cloud)

Problem: When deploying the Streamlit dashboard to Streamlit Cloud, the app crashed with a `ModuleNotFoundError` related to `sklearn._loss`. The root cause was a **scikit-learn version mismatch** — the models (`gradient_boosting.pkl`, `random_forest.pkl`) were trained locally with scikit-learn 1.x, but Streamlit Cloud was running a newer Python environment (Python 3.14 compatible) where the internal `_loss` module path had changed between versions. Pickle files are tightly coupled to the exact scikit-learn version they were trained with, making them unloadable across versions.

Error message: `ModuleNotFoundError: No module named 'sklearn._loss'`

Solution: Rather than continuously fighting scikit-learn version pinning issues on Streamlit Cloud, the decision was made to **use the React frontend as the primary production interface**. The React frontend calls the FastAPI backend which controls its own Python environment with a pinned `scikit-learn==1.6.1` in `api/requirements.txt`. This solved the issue completely. The Streamlit dashboard is retained as a reference/development tool only.

Issue 2: Hopsworks Feature Store — Dependency & Materialization Failures

Problem: The project initially used **Hopsworks** as the feature store (a popular MLOps platform). GitHub Actions workflows were configured to push engineered features to a Hopsworks feature group and read them back for training. This failed completely in production due to multiple compounding issues:

Errors encountered in GitHub Actions:

- ✗ `ModuleNotFoundError: No module named 'twofish'`
- ✗ `ModuleNotFoundError: No module named 'pyarrow'`
- ✗ `ModuleNotFoundError: No module named 'confluent_kafka'`
- ✗ No hudi properties found for feature group
- ✗ Cannot read feature group — data not materialized

Honest Assessment of Hopsworks Free Tier:

Hopsworks is designed for their paid enterprise cloud. The free tier has fundamental limitations for this use case:

- Data materialization is **slow and unreliable** — data pushed may not be readable for hours
- Requires **many heavy native dependencies** that are difficult to install in GitHub Actions CI environment
- The Hudi-based storage format requires complex setup not suited for lightweight pipelines
- Kafka-based streaming infrastructure is overkill and unavailable without paid plan

Solution: Hopsworks was completely removed and replaced with **MongoDB Atlas as the feature store**. MongoDB was already used in the project for raw data storage, making the migration straightforward:

- `aqi_features` collection → raw hourly data (replaces Hopsworks feature group)
- `aqi_engineered` collection → fully engineered features ready for training
- `model_metadata` collection → training run logs and metrics

Benefits of MongoDB over Hopsworks for this project:

- Zero dependency issues — uses pymongo (a simple Python package)
- Instant data availability — no materialization lag
- Works perfectly in GitHub Actions with certifi for SSL
- Free tier is genuinely usable with no hidden limitations
- Already part of the project stack

15. PROJECT REQUIREMENTS FULFILLMENT

Requirement	Status
Feature Pipeline	✓ Complete
Historical Data Backfill (3 months)	✓ Complete
Training Pipeline	✓ Complete
Multiple ML Models	✓ Complete (GB, RF, Ridge, Lasso, ElasticNet, LSTM)
RMSE, MAE, R ² Evaluation	✓ Complete
Automated CI/CD Pipeline	✓ Complete (GitHub Actions hourly + daily)
Web Dashboard (React + Streamlit)	✓ Complete
Feature Store (MongoDB)	✓ Complete
SHAP Explainability	✓ Complete
AQI Health Alerts	✓ Complete
Multiple Forecasting Models	✓ Complete
TensorFlow / Deep Learning (LSTM)	✓ Complete
FastAPI REST Backend	✓ Complete
GitHub Actions	✓ Complete
Cloud Deployment (Vercel + Render)	✓ Complete